

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

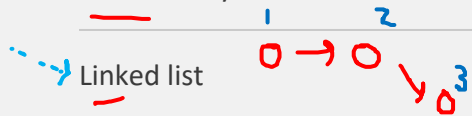
HASHING I

$O(n^2)$
↓
 $O(\log n)$
↓
 $O(1)$



WHERE HASHING SITS IN THE COURSE

Every other structure in this table has a lookup cost that grows with n . The hash table is the first one that appears not to.

STRUCTURE	SEARCH	INSERT	DELETE	ORDERED
<u>Unsorted array</u>	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	no
<u>Sorted array</u>	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	yes
 Linked list	$\Theta(n)$	<u>$\Theta(1)$</u>	$\Theta(n)$	<u>no</u>
Balanced BST	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	yes
<u>Hash table</u>	$\Theta(1)$ *	$\Theta(1)$ *	$\Theta(1)$ *	no

THE ASTERISK

Every other bound on this table is worst case. The hash table row is not. It is an expected cost, and it holds only under an assumption we have to state and then defend.

WHY WE SKIP THE BST

A balanced BST does every operation in $\Theta(\log n)$ worst case and keeps the keys sorted. That is all of it, so it does not get its own lecture.

Two questions decide between them. Do you need the keys in sorted order? Do you need a worst-case guarantee rather than an average? If either answer is yes, use a BST. If both are no, use a hash table, and both are no most of the time.



HASH FUNCTIONS AND HASH TABLES



THE DICTIONARY: KEY PROPERTIES

dict()

Maintain a collection of key-value pairs under three operations. Nothing in the specification says anything about order. That omission is what hashing exploits.

01 **insert(k, v)**

Associate the value v with the key k , replacing any value already stored under k .

02 **lookup(k)**

Return the value stored under k , or report that k is not present in the collection.

03 **delete(k)**

Remove the entry for k . Later lookups of k must report it as absent.

"whatever"

WHY THIS MATTERS

This is the most heavily used data structure interface in computing: symbol tables, database indexes, caches, memoization, deduplication, sparse matrices, adjacency lookups.

WHAT WE ARE COMPARING AGAINST

A balanced binary search tree does all three in $\Theta(\log n)$. At $n = 10^9$ that is roughly thirty comparisons per lookup, and the count grows with the data. We do not cover trees in this course, so take the bound as given.

The goal is a structure whose cost does not grow with n at all. Not slower growth. No growth at all.



DIRECT ADDRESSING IS THE ALGORITHM WE WANT

Suppose every key is an integer between 0 and $m-1$. Then the key is already an array index, and there is nothing left to design.

```
# keys are integers in 0 .. m-1
insert(k, v):  A[k] = v
lookup(k):    return A[k]
delete(k):    A[k] = None
```

Every operation is one array index: $O(1)$ worst case, not expected. No assumption, no probability, no collisions. This is the behaviour we are going to spend two lectures approximating.

0 ... m-1

WHY WE CANNOT HAVE IT

The array needs one slot for every key that could possibly appear, so it is sized by the universe U of possible keys, not by the number of keys n you actually store.

THE UNIVERSE IS ALWAYS ENORMOUS

64-bit integer keys give $|U| = 2^{64}$. Twenty-character ASCII strings give $|U| = 128^{20}$. Arbitrary strings give an unbounded universe. Meanwhile n might be a thousand.

2^{50} 254 — — —



THREE DEFINITIONS, THREE DIFFERENT OBJECTS

These three terms get used interchangeably in the wild. They are not interchangeable, and exam questions rely on the distinction.

HASH FUNCTION

A function $h : U \rightarrow \{0, 1, \dots, m-1\}$. It accepts any key in the universe U and returns a slot index in a table of size m . It is a function, not a data structure.

HASH VALUE $\rightarrow$ number

The output $h(k)$ for one particular key k . Also called the hash code, or informally the key's slot. It is a number.

HASH TABLE

An array of m slots together with a hash function h . Key k is stored at slot $h(k)$, or near it. It is the data structure.

A CONCRETE EXAMPLE

$$m = 7, \quad h(k) = k \bmod 7$$

KEY	COMPUTATION	SLOT
15	$15 \bmod 7 = 1$	1
3	$3 \bmod 7 = 3$	3
18	$18 \bmod 7 = 4$	4
25	$25 \bmod 7 = 4$	4 (taken)

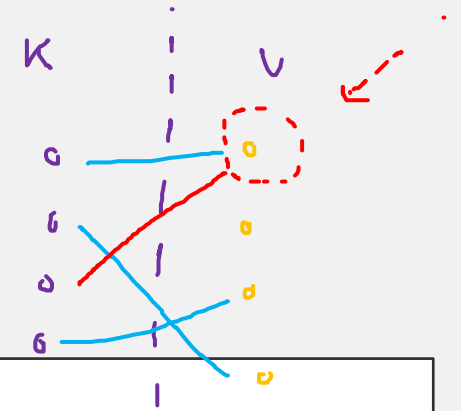
NOTATION FOR THE WHOLE UNIT

m = number of slots in the table
 n = number of keys currently stored
 U = universe of possible keys
 $\alpha = n / m$, the load factor



COLLISIONS ARE FORCED, NOT ACCIDENTAL

Two distinct keys landing in the same slot is called a collision. It is not a sign that the hash function is bad. It is a counting fact.



PIGEONHOLE PRINCIPLE

If $|U| > m$, then h cannot be injective: there must exist distinct keys $k_1 \neq k_2$ with $h(k_1) = h(k_2)$. Since $|U|$ is astronomically larger than m in every real application, every hash function has enormously many colliding pairs, and no choice of h changes that.

WHAT A GOOD h CAN DO

Uniform

Spread keys evenly, so that no slot receives many more than its share. This is achievable, and it is what the third module is about.

WHAT NO h CAN DO

Eliminate collisions on an unbounded universe. Handling them is therefore mandatory, not an optimisation you add later if profiling says so.

WHAT WE ASSUME TODAY

That some ^{..}black box^{..} resolves collisions correctly. We will treat the cost of that box as a parameter and fill it in next lecture.

The exam version of this point: a hash function is judged by its distribution, not by its collision count. Asking for zero collisions on an infinite universe is asking for an injection from an infinite set into a finite one.



OPERATIONS AND RUNTIME



EVERY OPERATION HAS THE SAME TWO-STEP SHAPE

This decomposition is what makes hash table analysis tractable: one step has a cost we control, and the other has a cost we have to reason about probabilistically.

```
# table: array of m slots
# h:      hash function into 0 .. m-1
```

```
insert(k, v):
```

```
    i = h(k)                # step 1
```

```
    place (k,v) at slot i    # step 2
```

```
lookup(k):
```

```
    i = h(k)                # step 1
```

```
    search slot i for key k # step 2
```

```
    return its value, or None
```

```
delete(k):
```

```
    i = h(k)
```

```
    remove key k from slot i
```

STEP 1: COMPUTE $H(K)$

Cost depends on the size of the key, never on n . For fixed-width keys it is $\Theta(1)$; for a string of length L it is $\Theta(L)$, which we will treat as constant unless a question says otherwise.

STEP 2: RESOLVE THE SLOT

Examine the keys already sitting at or near slot $h(k)$. This is the only step whose cost can grow with n , and it grows only when keys collide.

So the entire analysis reduces to one question: how many keys do we have to look at in step 2?



THE LOAD FACTOR IS THE PARAMETER THAT MATTERS

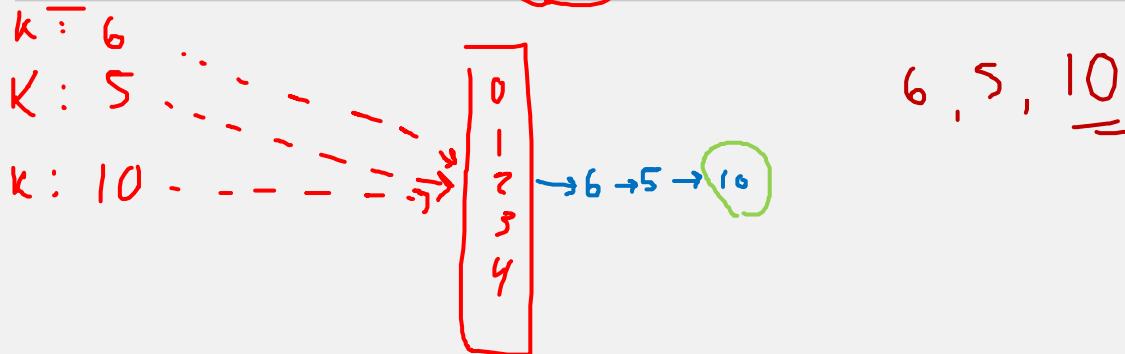
Every runtime bound in this unit is stated in terms of α , and never in terms of n by itself. That is not a stylistic choice. n alone does not determine the cost.

$$\alpha = n / m$$

keys stored $\div$ slots available



N	M	α	INTERPRETATION
5	10	0.50	half full
500 000	1 000 000	0.50	identical cost per lookup
750	1 000	0.75	the usual resize threshold
3 000	1 000	3.00	three keys per slot on average



WHAT α MEASURES

The average number of keys per slot. Rows one and two of the table have wildly different n and identical α , and identical expected lookup cost. That is the point.

HOW α IS HELD CONSTANT

Resize the table when α crosses a threshold, typically doubling m when α exceeds 0.75 and then re-placing every key. Because m grows with n , α never grows.

That mechanism is what turns a $\Theta(1 + \alpha)$ bound into a $\Theta(1)$ bound. Without it, "expected constant time" is simply false as n increases.



BEST, WORST, AND EXPECTED CASES

Using the case-analysis machinery from Week 1. Two of these three are unconditional facts. The third is a claim that depends on an assumption.

BEST CASE

$$\Theta(1)$$

The slot we hash to holds exactly the key we want, or is empty. One probe, no scanning. This happens whenever the key did not collide. ✓

WORST CASE

$$\Theta(n)$$

Every key in the table hashed to the same slot. The structure has degenerated into a single linked list, and lookup scans all n keys. $h(k) = k \% m$

EXPECTED

$$\Theta(1 + \alpha)$$

Averaged over the randomness in where keys land. This is the bound we actually rely on in practice, and the only one that requires an assumption.

SIMPLE UNIFORM HASHING ASSUMPTION (SUHA)

Each key is equally likely to hash to any of the m slots, independently of where the other keys hash. Formally: $\Pr[h(k) = j] = 1/m$ for every key k and every slot j .

The expectation above is taken with respect to this distribution. If the assumption fails, and an adversary can make it fail deliberately, the $\Theta(1)$ claim collapses back to $\Theta(n)$.

YPS



WHY THE HASH TABLE IS THE DEFAULT

Not because it is asymptotically unbeatable. A balanced BST is only a log factor behind. Because of the combination of properties below.

01 The cost does not grow with n

$\Theta(1 + \alpha)$, and resizing keeps α bounded by a constant.

02 The constant is genuinely small

One arithmetic operation and one or two memory accesses.

03 Keys can be anything hashable

Strings, tuples, objects. No ordering or comparison operator required.

04 Space is $\Theta(n)$, not $\Theta(|U|)$

Unlike direct addressing, space is proportional to the keys you actually store.

05 It is already in the language

Python dict and set, Java HashMap, C++ unordered_map, Go map.

WHERE YOU ARE ALREADY USING ONE

Membership testing and deduplication. Visited sets in BFS and DFS, which you will write in two weeks. Counting and group-by aggregation. Memoisation. Database indexes and joins. Compiler symbol tables. Sparse matrices.

WHEN IT IS THE WRONG CHOICE

You need sorted iteration, min or max, successor or predecessor, or range queries, in which case you want a balanced BST. Or you need a hard worst-case guarantee, and $\Theta(n)$ on an adversarial input is unacceptable.



WHAT MAKES A HASH FUNCTION GOOD



FOUR REQUIREMENTS

The first two are engineering hygiene. The third is the one the analysis depends on. The fourth is the one people actually get wrong.

01 Deterministic

$h(k)$ must return the same slot every single time it is called on k . If the value drifts, previously inserted keys become unreachable. This is why mutable objects cannot be dictionary keys in Python. A list's contents can change after insertion, so its hash would change with it. 

02 Fast to compute

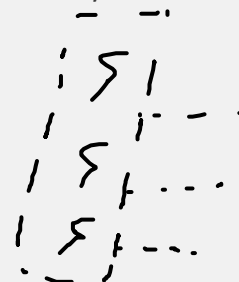
h must run in time proportional to the size of the key, not to the number of keys stored. A hash function that costs $\Theta(n)$ defeats the entire structure. You would be better off scanning an unsorted array.

03 Uniform

Keys should spread evenly across all m slots. This is the property that makes SUHA plausible, and therefore the property the $\Theta(1 + \alpha)$ bound depends on. It is also the only one of the four that is a statement about a distribution rather than about a single evaluation.

04 Uses the whole key

If h ignores part of the key, then any two keys differing only in the ignored part always collide. Taking h of a string to be its first character sends every Smith, Sanchez, and Singh to one slot, and surnames are not uniformly distributed to begin with.





UNIFORMITY, STATED FORMALLY

This is the assumption doing the work in every expected-cost bound in the unit. It is worth knowing exactly what it says and exactly where it is vulnerable.

Yes

SIMPLE UNIFORM HASHING ASSUMPTION

For every key k and every slot $j \in \{0, \dots, m-1\}$, $\Pr[h(k) = j] = 1/m$, and the slots of distinct keys are independent.

Equivalently: the m slots receive equal shares of the key universe, and knowing where k_1 landed tells you nothing about where k_2 will land.

WHAT IT GIVES US

Expected number of keys per slot is $n/m = \alpha$.

↙

Expected search cost is $\Theta(1 + \alpha)$, which we prove next lecture.

With $m = n$, the longest chain is $\Theta(\log n / \log \log n)$ with high probability, so even the unlucky slot is short.

WHY IT IS AN ASSUMPTION AND NOT A THEOREM

h is a fixed deterministic function. There is no randomness in $h(k)$ at all.

So the probability must be over the distribution of the keys, and we do not choose the keys.

Real key sets are structured, not uniform: sequential IDs, aligned pointers, common prefixes, timestamps at regular intervals.



THE DIVISION METHOD

$h(k) = k \bmod m$. One instruction, and the entire quality of the hash function comes down to the choice of m .

BAD: $M = 2^p$

$k \bmod 2^p$ is precisely the low p bits of k . Every bit above the p -th is discarded, so any structure in the high bits survives untouched, a direct violation of requirement four.

($p > 3$) . . .

GOOD: M PRIME, FAR FROM A POWER OF 2

If the keys arrive with a stride s , such as every fifth ID or every eighth byte address, then $k \bmod m$ visits only $m / \gcd(s, m)$ distinct slots. Choosing m prime forces that $\gcd$ to 1 whenever $s < m$, so all m slots stay reachable.

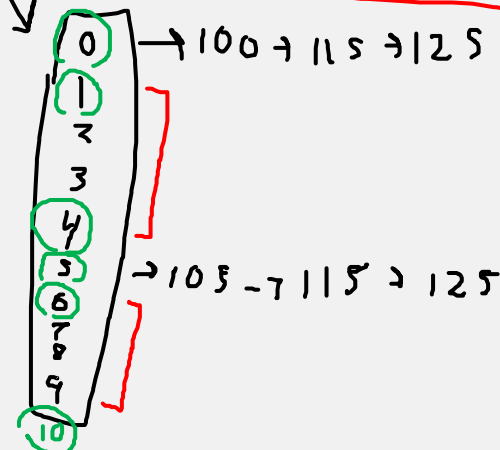
"31"

WORKED EXAMPLE: STUDENT IDS, ALL MULTIPLES OF 5

keys = 100, 105, 110, 115, 120, 125

M	SLOTS PRODUCED	DISTINCT	GCD(5, M)
10	0, 5, 0, 5, 0, 5	2	5
11	1, 6, 0, 5, 10, 4	6	1

Same keys, same method, one choice of m different. With $m = 10$ the table can never use more than two of its ten slots, no matter how many student IDs you insert. Eighty per cent of the memory is unreachable by construction.



THE MULTIPLICATION METHOD, AND HASHING STRINGS

Two constructions worth knowing: one that frees you from choosing m carefully, and one for the keys you will actually hash most often.

MULTIPLICATION METHOD

$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$ for a chosen constant $0 < A < 1$. Multiply the key by A , keep the fractional part, and scale it up to the table size.

WHY ANYONE USES IT

The value of m no longer matters, so powers of two are fine and the final scaling is a bit shift. Knuth's suggested constant is $A \approx (\sqrt{5} - 1)/2 = 0.6180339887$, which spreads keys well in practice. The tradeoff is that quality now depends on A instead of on m .

Fibonacci
hashing

```
# polynomial rolling hash for strings
h = 0
for ch in s:
    h = (h * x + ord(ch)) % m
return h
```

WHY NOT JUST ADD UP THE CHARACTERS

$h(s) = \sum \text{ord}(ch)$ makes every anagram collide: "listen" and "silent" land in the same slot. It also caps the output at $128 \cdot L$, so in a table with m larger than that, whole ranges of slots are unreachable. The positional weight x^i fixes both at once.

Anagrams

↔ silent ↔ listen ↔

ASCII

$\text{ord}('A') \rightarrow 65$
 $\text{ord}('a') \rightarrow 97$

62 % m

$O(n)$

120 /



ANY FIXED HASH FUNCTION HAS AN INPUT THAT DEFEATS IT

The previous two slides were about accidents. This one is about someone doing it on purpose, and it cannot be fixed by choosing a better h .

THE ARGUMENT

Fix any deterministic h . Since $|U| > m$, some slot j receives at least $|U|/m$ of the keys in the universe, by pigeonhole again. An adversary who knows h can enumerate that set and choose n keys that all map to j . Every lookup then costs $\Theta(n)$, and no amount of cleverness in the design of h prevents it, because the adversary chooses after seeing h .

adversary knows h ↑

THIS IS A REAL ATTACK, NOT A THOUGHT EXPERIMENT

Hash-flooding denial of service. Send a web server a request whose form fields are chosen to collide under the server's known string hash. The server's parameter dictionary degenerates into a linked list, and a request that should cost microseconds costs seconds. One machine can take down a service this way.

↓
hrs

THE FIX: MOVE WHERE THE RANDOMNESS LIVES

SUHA puts the randomness in the keys, which is exactly what the adversary controls.

Instead, put the randomness in the choice of h , which we control, and which we make at table creation time, after the adversary has committed to their keys. Python does this: PYTHONHASHSEED randomises string hashing per process.

$[h_1, h_2, h_3, \dots]$

↑



UNIVERSAL HASHING

The same guarantee as SUHA, obtained from randomness we control instead of an assumption about input we do not.

DEFINITION $[h_1, h_2, h_3, \dots, h_n]$

A family H of hash functions $U \rightarrow \{0, \dots, m-1\}$ is universal if for every pair of distinct keys $k \neq \ell$,

$\Pr \{h \in H \mid h(k) = h(\ell)\} \leq 1/m$, where the probability is over a uniformly random choice of h from H .

WHAT ACTUALLY CHANGED

The keys are now arbitrary, adversarial even. No distribution is assumed on them anywhere.

The probability is over our own coin flips, made once when the table is created.

So the bound holds for every input, and the expected $\Theta(1 + \alpha)$ cost survives an adversary who knows the algorithm but not our draw.

```
choose a prime  $p > |U|$ 
draw  $a$  in  $1..p-1$ ,  $b$  in  $0..p-1$  at random
 $h_{\{a,b\}}(k) = ((a*k + b) \bmod p) \bmod m$ 
```

Draw a and b once, when the table is built. The family $\{h_{\{a,b\}}\}$ is universal. We state that without proof; the argument turns on p being prime and is in any standard reference.



EXAM-STYLE

Three questions in the form the exam uses. Try them before the solutions on the next slide.

QUESTION 1

A table has $m = 8$ slots and uses $h(k) = k \bmod 8$. The keys inserted are 16, 24, 32, 40, and 48. How many distinct slots end up occupied? Explain your answer in terms of gcd, and state what choice of m would have avoided it.

QUESTION 2

True or false: a sufficiently well-designed hash function eliminates collisions. Justify your answer in one sentence, and name the principle you are appealing to.

QUESTION 3

A hash table stores $n = 3000$ keys in $m = 1000$ slots. Give the load factor, give the expected cost of an unsuccessful lookup under SUHA, and  explain why the Θ answer alone is misleading here.

All three are about the gap between what a bound says and what it means. Question 1 is arithmetic. Question 2 is a counting argument. Question 3 is about what Θ notation deliberately discards.



EXAM-STYLE SOLUTION

	ANSWER	REASONING
Q1	One slot. A chain of length 5. $m = 8$ $0 : 8 \rightarrow 16 \rightarrow 24$ $m = 7$	Every key is a multiple of 8, so $k \bmod 8 = 0$ for all of them. A stride s reaches $m/\gcd(s, m)$ slots; $\gcd(8, 8) = 8$, so $8/8 = 1$. Choosing $m = 7$ or 11 makes the gcd 1 and spreads the keys.
Q2	False. $0 : x$ $1 : x$ $2 : x$ $3 : x$ $m = 7$	$U > m$ forces collisions by pigeonhole. No function from a larger set to a smaller one is injective. A good hash function spreads the unavoidable collisions uniformly, which is a different and attainable goal.
Q3	$\alpha = 3$; expected cost $\Theta(1 + \alpha) = \Theta(4)$. $(1) h ?$ $(2) m ?$ $(3) a ?$	$\Theta(4)$ is $\Theta(1)$, so the bound is constant and correct. But it hides a factor of four against a table at $\alpha = 0.5$, exactly what a resize policy exists to control. Constant in n does not mean fast.

THE THREAD RUNNING THROUGH ALL THREE

A hash table's performance is not a property of the hash function alone, or of n alone. It is a property of the interaction between the key set, the choice of m , and the load factor. Every one of those three is something you choose.



CLOSING THOUGHTS

